【转载】第七讲——使用Unidbg进行DFA攻击

转载 (https://www.qinless.com/category/reprint) · 2022-04-01 · 560 次浏览

文章目录[隐藏]

- 转载 24 / 25
- 说说在前面
- 一、引言
- 二、案例
- 二、尾声

转载 (https://www.qinless.com/series/zhuanzai) 24 / 25

【转载】unidbg hook 使用大全 (https://www.qinless.com/670)

【转载】ida 基本使用 常用快捷键 (https://www.qinless.com/732)

【转载】Frida和调试器共存的问题 (https://www.qinless.com/937)

【转载】Charles抓包Android最佳流量转发APP (https://www.qinless.com/939)

【转载】安卓自定义代码调用系统函数并编译so记录 (https://www.qinless.com/1020)

【转载】jnitrace 手机卡死黑屏问题解决 (https://www.qinless.com/1169)

【转载】Unidbg SO 逆向入门实战教程一 OASIS (https://www.qinless.com/1173)

【转载】Unidbg SO 逆向入门实战教程二 calculateS (https://www.qinless.com/1177)

【转载】Unidbg SO 逆向入门实战教程三 V2-Sign (https://www.qinless.com/1181)

【转载】Unidbg SO 逆向入门实战教程四 mfw (https://www.qinless.com/1185)

【转载】Unidbg SO 逆向入门实战教程五 qxs (https://www.qinless.com/1188)

【转载】Unidbg SO 逆向入门实战教程六 s (https://www.qinless.com/1192)

【转载】Unidbg SO 逆向入门实战教程七 main (https://www.qinless.com/1194)

【转载】Unidbg SO 逆向入门实战教程八 文件读写 (https://www.qinless.com/1201)

【转载】Unidbg SO 逆向入门实战教程九 blackbox (https://www.qinless.com/1204)

【转载】Unidbg SO 逆向入门实战教程十 simpleSign (https://www.qinless.com/1209)

返回顶部

【转载】android app 加密参数分析研究 hash aes (https://www.qinless.com/1634)

【转载】第一讲——从黑盒攻击模型到白盒攻击模型 (https://www.qinless.com/1642)

【转载】第二讲——白盒加密攻击方法的选择 (https://www.qinless.com/1645)

【转载】第三讲——差分故障攻击的原理 (https://www.qinless.com/1647)

【转载】第四讲——差分故障攻击的工具链 (https://www.qinless.com/1653)

【转载】第五讲——使用源码进行DFA攻击 (https://www.qinless.com/1655)

【转载】第六讲——使用Frida进行DFA攻击 (https://www.qinless.com/1657)

【转载】第七讲——使用Unidbg进行DFA攻击

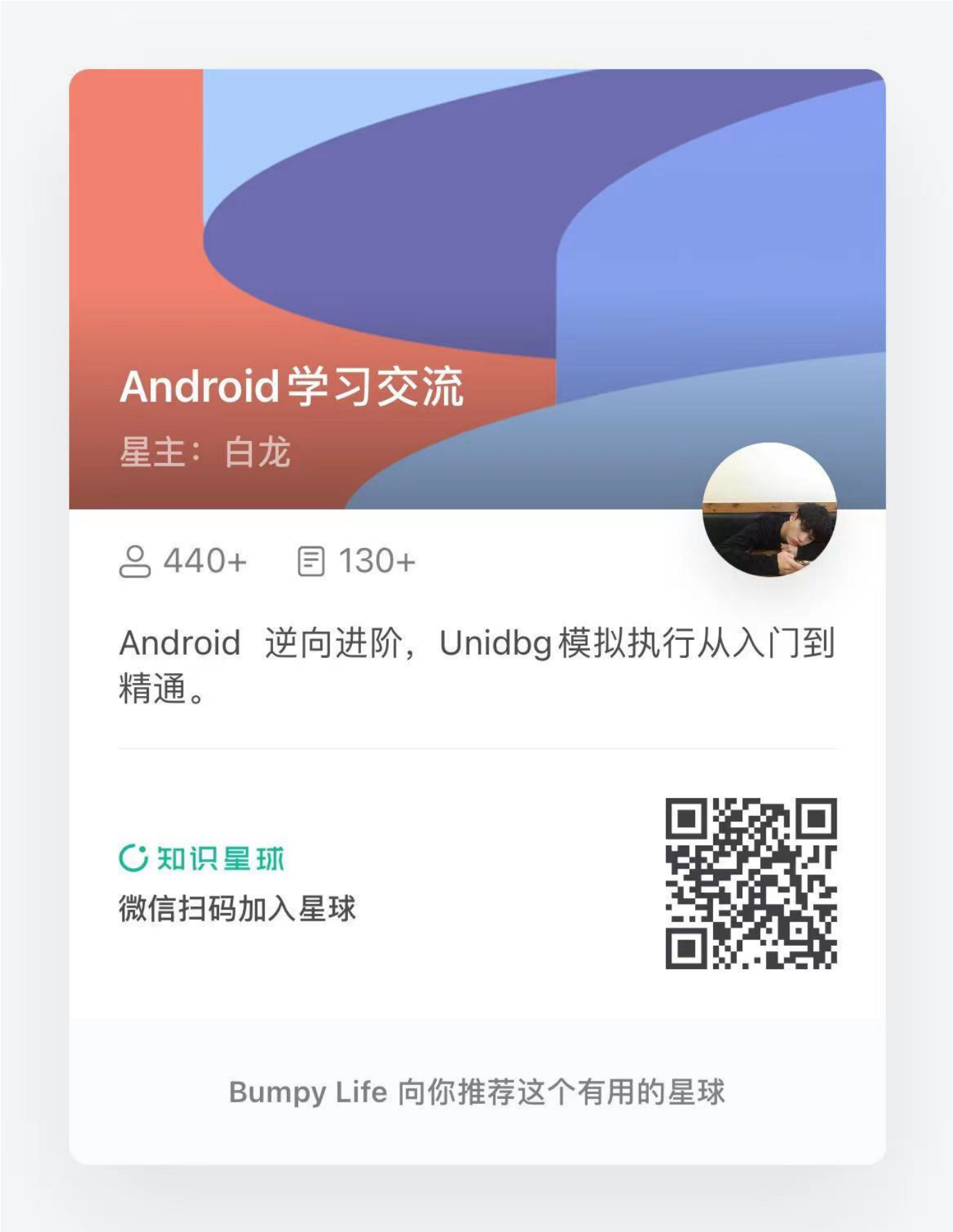
【转载】第八讲——案例 (https://www.qinless.com/1661)

« 上一篇文章 (https://www.qinless.com/1657)
下一篇文章 (https://www.qinless.com/1661) »

说说在前面

本文转载龙哥星球 白盒 AES 密码学系列，相关文件代码等，可加入星球下载

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/03/longge-zsxq.jpeg)

一、引言

返回顶部

这一篇中，我们简单讨论使用Unidbg处理DFA。

二、案例

案例就是前文所处理过的案例，在前面我们说，Unidbg中有另一个办法可以帮助”找时机“。我先卖个关子，大家先往下看。

首先和前文一样，Call wbaes_encrypt_ecb。

返回顶部

COPY

```
package com.luckin;

import com.github.unidbg.AndroidEmulator;
import com.github.unidbg.Module;
import com.github.unidbg.linux.android.AndroidEmulatorBuilder;
import com.github.unidbg.linux.android.AndroidResolver;
import com.github.unidbg.linux.android.dvm.AbstractJni;
import com.github.unidbg.linux.android.dvm.DalvikModule;
import com.github.unidbg.linux.android.dvm.VM;
import com.github.unidbg.memory.Memory;
import com.github.unidbg.memory.MemoryBlock;
import com.github.unidbg.pointer.UnidbgPointer;

import java.io.File;

public class LKAES extends AbstractJni {
    private final AndroidEmulator emulator;
    private final VM vm;
    private final Module module;

    public LKAES(){
        emulator = AndroidEmulatorBuilder.for32Bit().build(); // 创建模拟器实例，要模拟32位或者

        // 模拟器的内存操作接口
        final Memory memory = emulator.getMemory();
        // 设置系统类库解析
        memory.setLibraryResolver(new AndroidResolver(23));
        // 创建Android虚拟机
        vm = emulator.createDalvikVM(new File("unidbg-android/src/test/resources/luckin/ruixing
        vm.setVerbose(true);
        // 加载so到虚拟内存
        DalvikModule dm = vm.loadLibrary("cryptoDD", true);
        module = dm.getModule();
        // 设置JNI
        vm.setJni(this);
        dm.callJNI_OnLoad(emulator);
    }

    public static byte[] hexStringToBytes(String hexString) {
        if (hexString.isEmpty()) {
            return null;
        }
        hexString = hexString.toLowerCase();
```

返回顶部

```
    final byte[] byteArray = new byte[hexString.length() >> 1];
    int index = 0;
    for (int i = 0; i < hexString.length(); i++) {
        if (index  > hexString.length() - 1) {
            return byteArray;
        }
        byte highDit = (byte) (Character.digit(hexString.charAt(index), 16) & 0xFF);
        byte lowDit = (byte) (Character.digit(hexString.charAt(index + 1), 16) & 0xFF);
        byteArray[i] = (byte) (highDit << 4 | lowDit);
        index += 2;
    }
    return byteArray;
}

public static String bytesTohexString(byte[] bytes) {
    StringBuffer sb = new StringBuffer();
    for(int i = 0; i < bytes.length; i++) {
        String hex = Integer.toHexString(bytes[i] & 0xFF);
        if(hex.length() < 2){
            sb.append(0);
        }
        sb.append(hex);
    }
    return sb.toString();
}

public void call_wbaes(){
    MemoryBlock inblock = emulator.getMemory().malloc(16, true);
    UnidbgPointer inPtr = inblock.getPointer();
    MemoryBlock outblock = emulator.getMemory().malloc(16, true);
    UnidbgPointer outPtr = outblock.getPointer();
    byte[] stub = hexStringToBytes("303132333435363738396162636465 66");
    assert stub != null;
    inPtr.write(0, stub, 0, stub.length);
    module.callFunction(emulator, 0x17bd5, inPtr, 16, outPtr, 0);
    String ret = bytesTohexString(outPtr.getByteArray(0, 0x10));
    System.out.println("white box result:"+ret);
    inblock.free();
    outblock.free();
}

public static void main(String[] args) {
    LKAES lkaes = new LKAES();
```

返回顶部

```
        lkaes.call_wbaes();
    }
}
```

运行结果

```
[15:41:39 547]  INFO [com.github.unidbg.linux.AndroidElfLoader] (AndroidElfLoader:464) - libcry
JNIEnv->FindClass(com/luckincoffee/safeboxlib/CryptoHelper) was called from RX@0x4001b43b[libcr
JNIEnv->RegisterNatives(com/luckincoffee/safeboxlib/CryptoHelper, RW@0x400dfd0c[libcryptoDD.so]
RegisterNative(com/luckincoffee/safeboxlib/CryptoHelper, localAESWork([BI[B)[B, RX@0x4001984d[l
RegisterNative(com/luckincoffee/safeboxlib/CryptoHelper, localConnectWork([B[B)[B, RX@0x4001978
RegisterNative(com/luckincoffee/safeboxlib/CryptoHelper, md5_crypt([BI)[B, RX@0x4001a981[libcry
RegisterNative(com/luckincoffee/safeboxlib/CryptoHelper, localAESWork4Api([BI)[B, RX@0x4001b1cd
white box result:b0f59c0d48c145915fc8f6a842c4d5eb
```

可以发现计算结果完全一致。

接下来我们要作图，读者可能会困惑，做什么图？事实上，因为白盒加密的主要实现方式是查表法，所以加密主体就是大量的内存访问。那么记录函数对内存的访问以及发起访问的地址（PC指针），绘制成折线图，就可以较好的反映加密流程。

使用Unidbg的ReadHook

返回顶部

```
public void traceAESRead(){
    emulator.getBackend().hook_add_new(new ReadHook() {
        @Override
        public void hook(Backend backend, long address, int size, Object user) {
            long now = emulator.getBackend().reg_read(ArmConst.UC_ARM_REG_PC).intValue();
            if((now>module.base) & (now < (module.base+module.size))){
                System.out.println(now - module.base);
            }
        }

        @Override
        public void onAttach(UnHook unHook) {

        }

        @Override
        public void detach() {

        }
    }, module.base, module.base+module.size, null);
}
```

规则如下：监控整个SO地址范围内的内存读取操作，记录其发起地址，我减去了SO基地址，只打印偏移，这样呈现效果更好。

traceAESRead 函数放于如下位置

```
public static void main(String[] args) {
    LKAES lkaes = new LKAES();
    lkaes.traceAESRead();
    lkaes.call_wbaes();
}
```

运行结果如图

返回顶部

(https://www.qinless.com/wp-content/uploads/2022/04/6247029eddea3.png)

将这几千条记录拷贝出来，保存在trace.txt 中，在Python中做可视化，这十分方便。需要安装matplotlib以及numpy库。

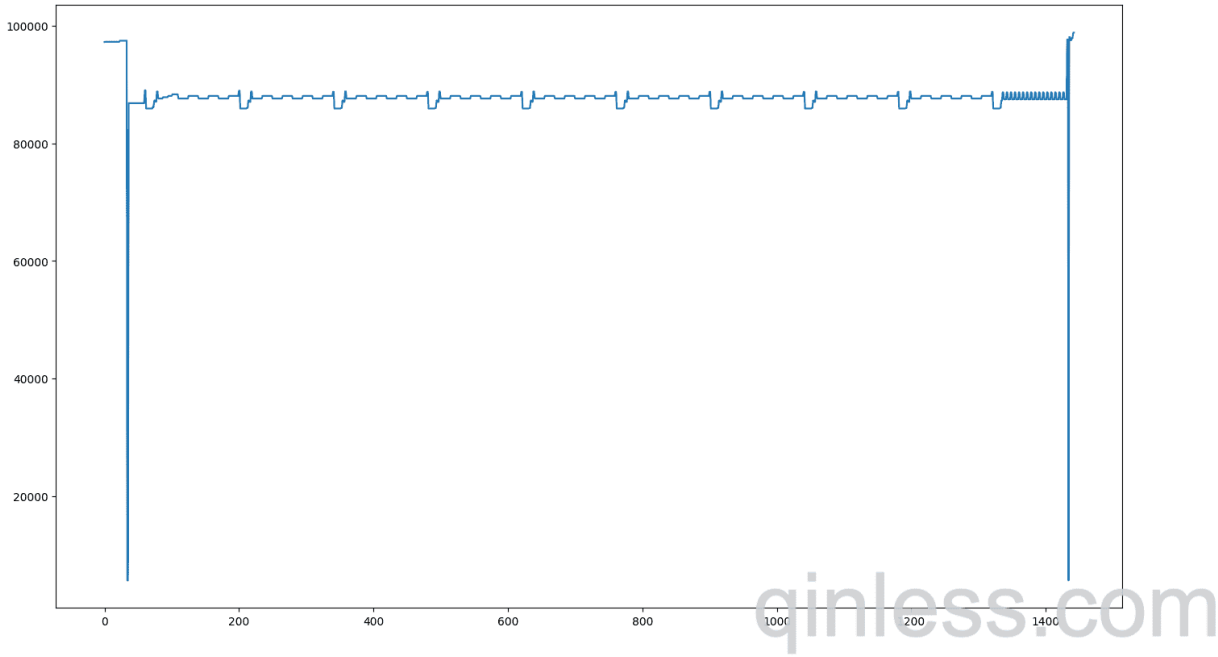
```
import matplotlib.pyplot as plt
import numpy

input = numpy.loadtxt("trace.txt", int)

plt.plot(range(len(input)), input)
plt.show()
```

运行后生成折线图，将其放大是如下效果

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/624702ab06ad8.png)

X轴的计数单位是次数，表示当前是第几次内存访问，如图，在程序的运行过程中，发生了1400余次对SO内存的读操作。Y轴是发起访问的偏移地址。需要注意，X与Y轴的数值表示为十进制。图上可得，Y主要在80000-100000之间，我们修改Y轴范围，增强呈现效果。

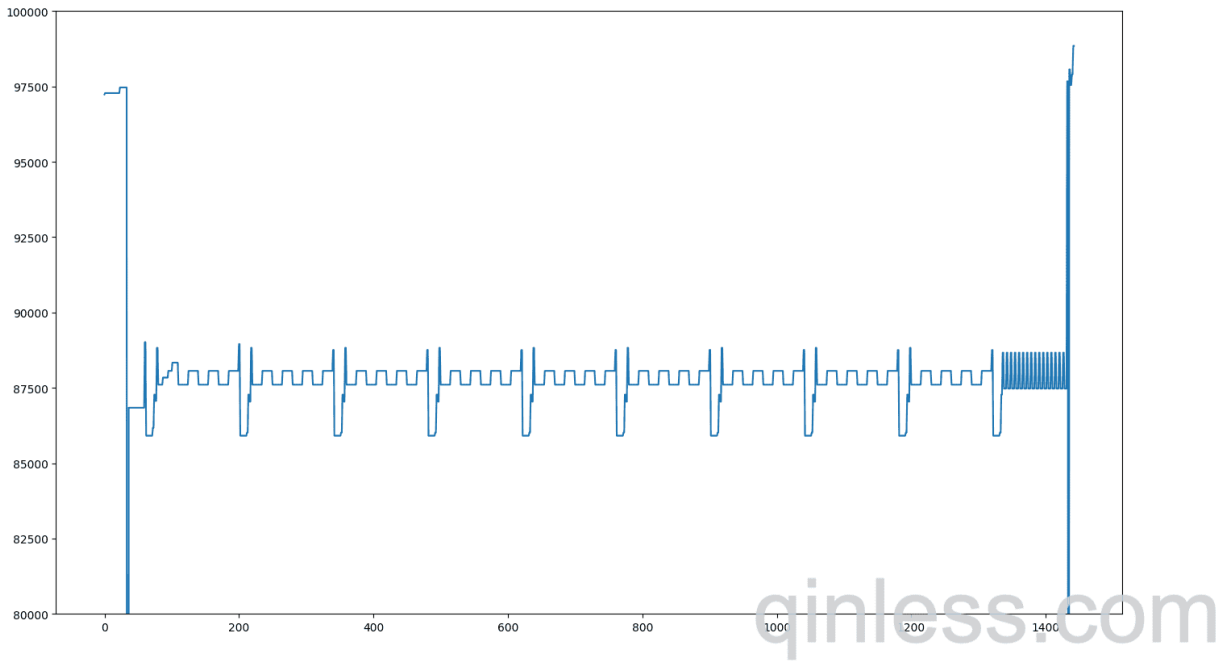
```
import matplotlib.pyplot as plt
import numpy

input = numpy.loadtxt("trace.txt", int)

# 限制Y
plt.ylim(80000,100000)
plt.plot(range(len(input)), input)
plt.show()
```

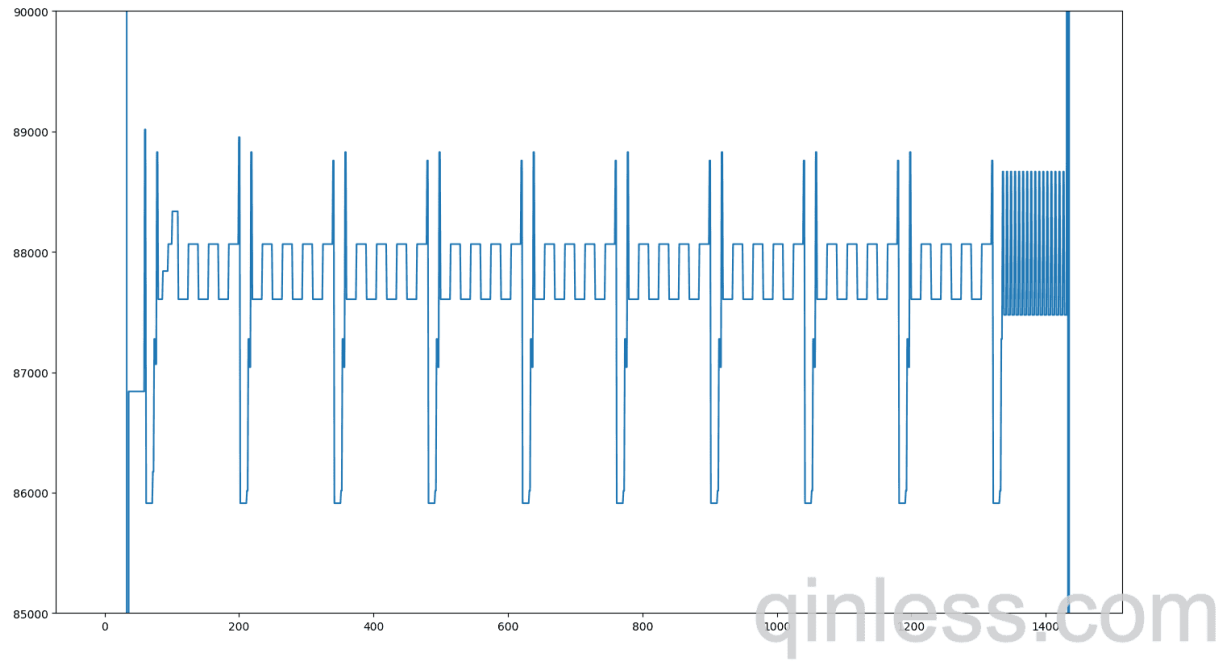
运行后

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/624702b8bfbcb.png)

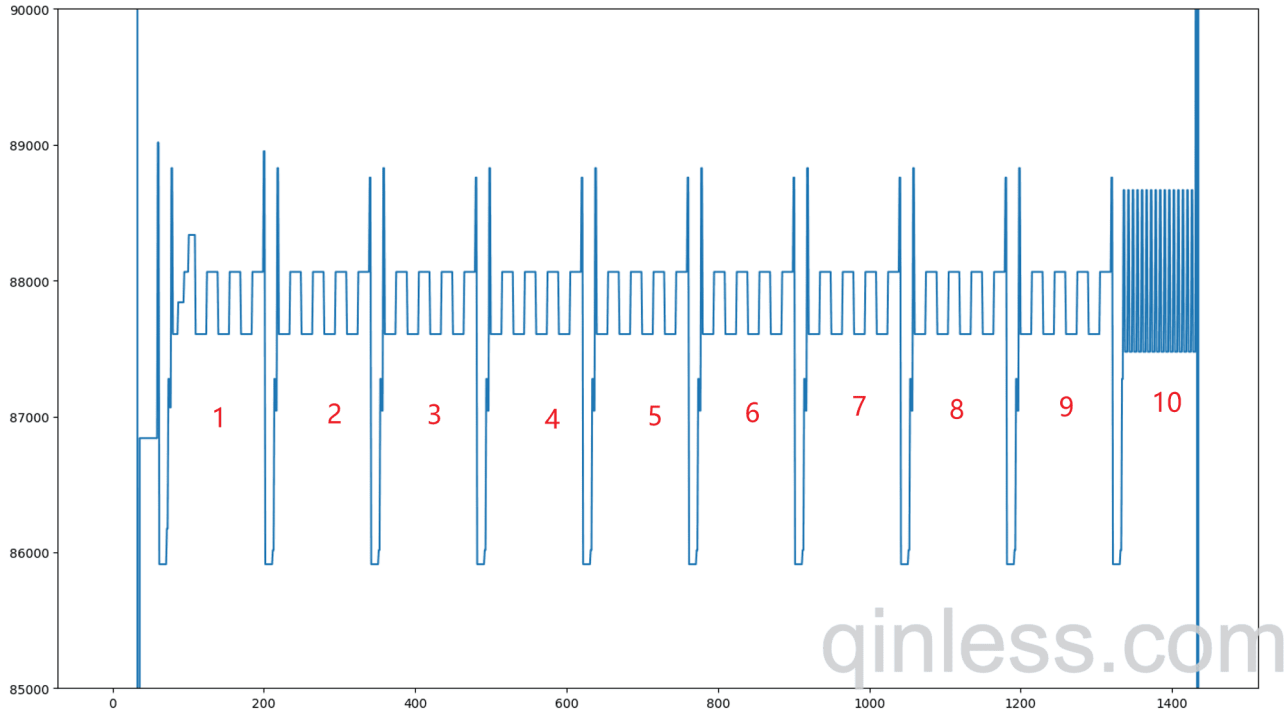
似乎还可以缩小到85000-90000之间，再次缩小Y的范围



(https://www.qinless.com/wp-content/uploads/2022/04/624702c3c6f84.png)

这样看着顺眼多了，我们对它进行分析。

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/04/624702ce19324.png)

首先，可以比较明显的看到，存在十个重复的模式，这代表了十轮运算。这一点是有用的，可用于区分AES-128/192/256，分别对应10/12/14 轮。

除此之外，我们发现每轮运算的起点是一个较低的地址，具体在86000附近左右，转成十六进制就是0x14FF0附近。

在IDA 中查看该处，我们发现正是上一篇中所分析的wbShiftRows中。这就是我说的”Unidbg中的另一个好办法“，这得益于Unidbg中获取执行流非常容易。

返回顶部

```
1  void __fastcall wbShiftRows(uint8_t *out)
2  {
3    int v1; // r4
4    int v2; // r5
5    uint8_t v3; // r9
6    uint8_t v4; // r8
7    uint8_t v5; // r12
8    uint8_t v6; // r5
9    uint8_t v7; // r10
10   uint8_t v8; // r11
11   uint8_t v9; // r4
12   uint8_t v10; // r5
13   uint8_t v11; // r9
14   uint8_t v12; // r4
15   uint8_t v13; // r11
16   uint8_t v14; // r10
17   uint8_t v15; // r12
18   uint8_t v16; // r8
19   uint8_t v17; // r9
20   uint8_t v18; // r5
21   uint8_t v19; // r12
22   int v20; // r8
23   uint8_t v21; // r4
24   unsigned int v22; // r5
25   unsigned int v23; // r12
26   int v24; // r5
27   bool v25; // [sp+Ah] [bp-1Eh]
28   bool v26; // [sp+Bh] [bp-1Dh]
29
30   v26 = y < 10;
31   v25 = (((x * (x - 1)) ^ 0xFFFFFFFE) & (x * (x - 1))) == 0;
32   v1 = 1187476278;
33   while ( 1 )
34   {
35     while ( v1 <= 761114409 )
36     {
37       if ( v1 == -1241595523 )
38       {
39         v3 = out[1];
40         v4 = out[2];
41         v5 = out[3];
```

qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/624702db0efaf.png)

接下来就再次来到故障攻击的部分，整个代码近似于Frida Hook代码的复刻。

返回顶部

COPY

```
package com.luckin;

import com.github.unidbg.AndroidEmulator;
import com.github.unidbg.Emulator;
import com.github.unidbg.Module;
import com.github.unidbg.arm.backend.Backend;
import com.github.unidbg.arm.backend.ReadHook;
import com.github.unidbg.arm.backend.UnHook;
import com.github.unidbg.arm.context.RegisterContext;
import com.github.unidbg.debugger.BreakPointCallback;
import com.github.unidbg.linux.android.AndroidEmulatorBuilder;
import com.github.unidbg.linux.android.AndroidResolver;
import com.github.unidbg.linux.android.dvm.AbstractJni;
import com.github.unidbg.linux.android.dvm.DalvikModule;
import com.github.unidbg.linux.android.dvm.VM;
import com.github.unidbg.memory.Memory;
import com.github.unidbg.memory.MemoryBlock;
import com.github.unidbg.pointer.UnidbgPointer;
import unicorn.ArmConst;

import java.io.File;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Random;

public class LKAES extends AbstractJni {
    private final AndroidEmulator emulator;
    private final VM vm;
    private final Module module;

    public LKAES(){
        emulator = AndroidEmulatorBuilder.for32Bit().build(); // 创建模拟器实例，要模拟32位或者

        // 模拟器的内存操作接口
        final Memory memory = emulator.getMemory();
        // 设置系统类库解析
        memory.setLibraryResolver(new AndroidResolver(23));
        // 创建Android虚拟机
        vm = emulator.createDalvikVM(new File("unidbg-android/src/test/resources/luckin/ruixing
        vm.setVerbose(true);
        // 加载so到虚拟内存
        DalvikModule dm = vm.loadLibrary("cryptoDD", true);
        module = dm.getModule();
```

返回顶部

```
        // 设置JNI
        vm.setJni(this);
        dm.callJNI_OnLoad(emulator);
    }

    public static byte[] hexStringToBytes(String hexString) {
        if (hexString.isEmpty()) {
            return null;
        }
        hexString = hexString.toLowerCase();
        final byte[] byteArray = new byte[hexString.length() >> 1];
        int index = 0;
        for (int i = 0; i < hexString.length(); i++) {
            if (index  > hexString.length() - 1) {
                return byteArray;
            }
            byte highDit = (byte) (Character.digit(hexString.charAt(index), 16) & 0xFF);
            byte lowDit = (byte) (Character.digit(hexString.charAt(index + 1), 16) & 0xFF);
            byteArray[i] = (byte) (highDit << 4 | lowDit);
            index += 2;
        }
        return byteArray;
    }

    public static String bytesTohexString(byte[] bytes) {
        StringBuffer sb = new StringBuffer();
        for(int i = 0; i < bytes.length; i++) {
            String hex = Integer.toHexString(bytes[i] & 0xFF);
            if(hex.length() < 2){
                sb.append(0);
            }
            sb.append(hex);
        }
        return sb.toString();
    }

    public void call_wbaes(){
        MemoryBlock inblock = emulator.getMemory().malloc(16, true);
        UnidbgPointer inPtr = inblock.getPointer();
        MemoryBlock outblock = emulator.getMemory().malloc(16, true);
        UnidbgPointer outPtr = outblock.getPointer();
        byte[] stub = hexStringToBytes("303132333435363738396162636465 66");
        assert stub != null;
```

返回顶部

```
        inPtr.write(0, stub, 0, stub.length);
        module.callFunction(emulator, 0x17bd5, inPtr, 16, outPtr, 0);
        String ret = bytesTohexString(outPtr.getByteArray(0, 0x10));
        System.out.println(ret);
        inblock.free();
        outblock.free();
    }

    public static int randInt(int min, int max) {
        Random rand = new Random();
        return rand.nextInt((max - min) + 1) + min;
    }

    public void dfaAttack(){
        emulator.attach().addBreakPoint(module.base + 0x14F98 + 1, new BreakPointCallback() {
            int count = 0;
            UnidbgPointer pointer;
            @Override
            public boolean onHit(Emulator<?> emulator, long address) {
                count += 1;
                RegisterContext registerContext = emulator.getContext();
                pointer = registerContext.getPointerArg(0);
                emulator.attach().addBreakPoint(registerContext.getLRPointer().peer, new BreakP
                    @Override
                    public boolean onHit(Emulator<?> emulator, long address) {
                    if(count % 9 == 0){
                        pointer.setByte(randInt(0, 15), (byte) randInt(0, 0xff));
                    }
                    return true;
                    }
                });

                return true;
            }
        });
    }

    public void traceAESRead(){
        emulator.getBackend().hook_add_new(new ReadHook() {
            @Override
            public void hook(Backend backend, long address, int size, Object user) {
                long now = emulator.getBackend().reg_read(ArmConst.UC_ARM_REG_PC).intValue();
                if((now>module.base) & (now < (module.base+module.size))){
```

返回顶部

```
                    System.out.println(now - module.base);
                }
            }

            @Override
            public void onAttach(UnHook unHook) {

            }

            @Override
            public void detach() {

            }
        }, module.base, module.base+module.size, null);
    }

    public static void main(String[] args) {
        LKAES lkaes = new LKAES();
//        lkaes.traceAESRead();
        lkaes.dfaAttack();
        for(int i =0;i<200;i++){
            lkaes.call_wbaes();
        }

    }
}
```

将正确密文与运行出的故障密文放入phoenixAES以及stark，可以得到和前文相同的结果。

二、尾声

后续会再补充三个左右样例。

标签: # aes, # dfa, # unidbg, # 攻击模型, # 白盒aes (https://www.qinless.com/tag/baiheaes)

(https://www.qinless.com/1659)
(https://www.qinless.com/wp-content/themes/honey/public/qrcode?data=https://www.qinless.com/1659)
返回顶部
(https://www.qinless.com/1659) 暂无评论 (https://www.qinless.com/1659#respond)

【转载】第六讲——使用Frida进行DFA攻击
上一篇
(https://www.qinless.com/1657)

【转载】第八讲——案例
下一篇
(https://www.qinless.com/1661)

会爬山的小脑虎 (https://www.qinless.com/author/xiayu)
小爬爬

本文作者： 会爬山的小脑虎 (https://www.qinless.com/author/xiayu)
本文链接： https://www.qinless.com/?p=1659 (https://www.qinless.com/1659)
版权声明：本博客所有文章除特别声明外，均采用 CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/) 许可协议。转载请注明来自 qinless 的博客！ (https://www.qinless.com)

微信搜一搜
逆向小友

发表评论

昵称 *

邮箱 *

返回顶部

网址

输入评论内容...

提交评论

Copyright 2022. All Rights Reserved. Powered By XinTheme (http://www.xintheme.com) + WordPress 果酱 (https://blog.wpjam.com/) 浙ICP备2021029793号 (http://www.miibeian.gov.cn/)

返回顶部